

Plan de Pruebas — PaperBoost V1.0

1. Introducción

1.1 Propósito

Este documento define la estrategia, alcance y ejecución de las pruebas del sistema PaperBoost V1.0. Establece los criterios de aceptación y los tipos de validación aplicados a cada capa de la arquitectura.

1.2 Alcance

El plan cubre las 4 capas de la arquitectura:

- **Datos:** Modelos, repositorios (abstractos + InMemory), seguridad (PasswordHasher)
- **Lógica:** Controllers, services, validators, observers, session manager
- **Presentación:** Páginas (login, register, home_shell, inventory, product_form, sales), widgets (ProductCard)
- **Integración:** Rutas, navegación, composición de dependencias

1.3 Herramientas

Herramienta	Versión	Uso
Flutter SDK	3.x	Framework de desarrollo
flutter_test	(bundled)	Framework de pruebas
test	(bundled)	Paquete de pruebas unitarias
mockito	(bundled)	Mocking de dependencias
Dart	3.x	Lenguaje de programación

1.4 Criterios de Aceptación

- Todas las pruebas deben pasar sin errores
 - No se permiten pruebas saltadas (skip)
 - Cobertura mínima del 80% por capa
 - Cada cambio de código debe pasar el suite completo de pruebas
-

2. Estrategia de Pruebas

2.1 Niveles de Prueba

Nivel	Enfoque	Archivos	Cantidad
Unitaria	Funciones aisladas, modelos, repositorios	test/data/, test/logic/	~292
Widget	Componentes de UI, páginas, interacciones	test/presentation/	~136
Integración	Rutas, navegación, composición	test/main_test.dart	~20
Total			~428

2.2 Tipos de Prueba por Capa

Capa de Datos

- **Modelos:** Serialización (toMap/fromMap), copyWith, constructores factory, getters computados
- **Repositorios:** CRUD completo, validación de duplicados, manejo de errores (StateError)
- **Seguridad:** verificación

Capa de Lógica

- **Controllers:** Delegación correcta al service correspondiente, retorno de OperationResult
- **Services:** Validación de reglas de negocio, orquestación de repositorios, cálculos (IVA, stock)
- **Validators:** Reglas de validación por campo, mensajes de error esperados
- **Observers:** Notificación a observadores, attach/detach, clearObservers
- **SessionManager:** Login/logout, estado de autenticación, singleton

Capa de Presentación

- **Páginas:** Renderizado de widgets, interacciones de usuario, navegación
- **Widgets:** Props correctas, estados visuales (activo/inactivo, crítico)
- **Formularios:** Validación de campos, envío de datos, retroalimentación

2.3 Criterios de Parada

- Falla en pruebas críticas (login, registro, CRUD de productos/ventas)
- Error de excepción no manejada en repositorios

- Regresión en funcionalidad previamente aprobada

3. Plan de Pruebas por Archivo

3.1 Data — Modelos

Archivo	Pruebas	Enfoque
product_test.dart	15	Serialización, copyWith, getters, enums
app_user_test.dart	9	Guest factory, isGuest, serialización
sale_test.dart	12	Estado, cálculos, serialización
sale_item_test.dart	7	Subtotal, serialización
stock_alert_test.dart	10	Estado, serialización, copyWith

3.2 Data — Repositorios

Archivo	Pruebas	Enfoque
in_memory_product_repository_test.dart	19	CRUD, ordenamiento, SKU duplicado
in_memory_user_repository_test.dart	6	Usuario default, restricción único usuario
in_memory_sale_repository_test.dart	8	CRUD, filtrado por estado, cancelación
in_memory_stock_alert_repository_test.dart	5	CRUD, duplicado por producto

3.3 Data — Seguridad

Archivo	Pruebas	Enfoque
password_hasher_test.dart	10	Determinismo, verificación, diferenciación

3.4 Lógica — Controllers

Archivo	Pruebas	Enfoque
auth_controller_test.dart	6	Login, registro, logout, guest
product_controller_test.dart	6	CRUD delegado, búsqueda

Archivo	Pruebas	Enfoque
sale_controller_test.dart	6	Crear, completar, cancelar, buscar
stock_alert_controller_test.dart	6	CRUD, toggle, triggered alerts

3.5 Lógica — Services

Archivo	Pruebas	Enfoque
auth_service_test.dart	14	Login, registro, validación, guest
product_service_test.dart	18	CRUD, búsqueda, validación, SKU duplicado
sale_service_test.dart	14	Crear venta, stock, cancelación, IVA
sale_search_test.dart	8	Búsqueda por query, estado
stock_alert_service_test.dart	10	CRUD, toggle, triggered alerts

3.6 Lógica — Validators

Archivo	Pruebas	Enfoque
auth_validator_test.dart	5	Email regex, password longitud
product_validator_test.dart	8	Campos obligatorios, precio, stock
sale_validator_test.dart	10	Items, stock, cliente, pago
stock_alert_validator_test.dart	3	Campos obligatorios, cantidad negativa

3.7 Lógica — Observers & Session

Archivo	Pruebas	Enfoque
stock_change_notifier_test.dart	6	Attach, notify, detach, clear
session_manager_test.dart	6	Singleton, login, logout, estado
operation_result_test.dart	4	Success, failure, data

3.8 Presentación — Páginas

Archivo	Pruebas	Enfoque
login_page_test.dart	3	Renderizado, login, guest
register_page_test.dart	3	Renderizado, registro, errores
home_shell_test.dart	11	Navegación, tabs, logout
inventory_page_test.dart	8	Renderizado, filtros, alertas, CRUD
product_form_page_test.dart	8	Formulario, validación, guardar
sales_page_test.dart	25	Crear venta, historial, detalle, cancelar

3.9 Presentación — Widgets

Archivo	Pruebas	Enfoque
product_card_test.dart	5	Renderizado, estado activo/inactivo, crítico

3.10 Integración

Archivo	Pruebas	Enfoque
main_test.dart	20	Rutas, navegación, composición
widget_test.dart	1	Smoke test de MaterialApp

4. Datos de Prueba

4.1 Productos de Ejemplo

- SKU: PROD-001, Nombre: Papel Bond A4, Precio: \$5.50, Stock: 100, Categoría: Papelería
- SKU: PROD-002, Nombre: Lapicero Bic, Precio: \$0.75, Stock: 200, Categoría: Útiles
- SKU: PROD-003, Nombre: Cartulina Blanca, Precio: \$1.20, Stock: 3, Categoría: Papelería

4.2 Usuarios de Ejemplo

- Admin: admin@paperboost.com / Admin123*
- Guest: (acceso sin credenciales)
- Registro: user@test.com / Test123*

4.3 Ventas de Ejemplo

- VTA-000001: 2x Papel Bond + 1x Lapicero, Efectivo, \$12.65 (IVA incluido)
- VTA-000002: 5x Cartulina, Tarjeta, \$6.90 (IVA incluido)

5. Ejecución

5.1 Comando de Ejecución

```
cd paperboost_V1.0
flutter test
```

5.2 Resultado Esperado

```
428 tests passed, 0 failed, 0 skipped
Execution time: ~35 seconds
```

6. Riesgos y Mitigación

Riesgo	Impacto	Mitigación
Tests de UI frágiles (tap fuera de pantalla)	Medio	Se emiten warnings pero pasan; se puede ajustar warnIfMissed
Cambios en modelos rompen serialización	Alto	Tests de toMap/fromMap cubren cada modelo
Estado singleton en tests	Medio	Se usa clearObservers() y se crean instancias fresh en setUp
Deprecaciones de Flutter	Bajo	Se actualizan imports según versiones

7. Aprobación

Rol	Nombre	Fecha
Líder de Equipo	Viscaino Jordy	Julio 2026
Desarrollador	Lucas Gongora	Julio 2026
Desarrollador	Klever Jami	Julio 2026